PART 2: DQN AGENT

Deep Q-Network for CartPole-v1

Gymnasium Environment · Discrete Action Space · MLP Neural Network

Reinforcement Learning Project Report

Academic Year 2024 / 2025

Environment:	CartPole-v1 (Gymnasium 0.29.1)
Observation:	RAM-based (4 continuous values)
Network type:	Fully-connected MLP
Framework:	PyTorch
Training eps.:	500 episodes

Team Members

Zeina Mohamed	ID: 202300450
Asmaa Sabry	ID: 202201338
Raed Asham	ID: 202200272

Contents

1 Task 1 – Game Description: CartPole-v1 **2**
1.1 Game Overview . . . 2
1.2 Environment Details . . . 2
1.3 Observation Vector (State) . . . 2
1.4 Action Space . . . 3
1.5 Why CartPole-v1 is Suitable for DQN . . . 3

2 Task 2 – DQN Components **3**
2.1 Policy Network . . . 3
2.2 Target Network . . . 3
2.3 Replay Buffer (Experience Replay) . . . 4
2.4 Epsilon-Greedy Exploration . . . 4
2.5 Action Selection Summary . . . 4

3 Task 3 – Neural Network Architecture **4**
3.1 Architecture Overview . . . 4
3.2 Layer-by-Layer Description . . . 5
3.3 Design Choices . . . 5
3.4 PyTorch Implementation . . . 5
3.5 Parameter Count . . . 6

4 Task 4 – Training the DQN Agent **6**
4.1 Training Loop Overview . . . 6
4.2 Bellman Target Calculation . . . 6
4.3 Loss Function and Optimisation . . . 7
4.4 Training Pseudocode . . . 7
4.5 Full Python Implementation . . . 7

5 Task 5 – Results and Discussion **8**
5.1 Hyperparameters Used . . . 9
5.2 Epsilon Decay Schedule . . . 9
5.3 Expected Results . . . 9
5.4 Training Curve Description . . . 10
5.4.1 Phase 1: Exploration (Episodes 1–100) . . . 10
5.4.2 Phase 2: Rapid Learning (Episodes 100–300) . . . 10
5.4.3 Phase 3: Convergence (Episodes 300–500) . . . 10
5.5 Discussion . . . 10
5.5.1 What Worked Well . . . 10
5.5.2 Challenges and Observations . . . 10
5.5.3 Observation Type . . . 11
5.6 Project Files . . . 11
5.7 How to Run . . . 11

References **11**

1 Task 1 – Game Description: CartPole-v1

1.1 Game Overview

CartPole-v1 is a classic control benchmark from OpenAI Gymnasium. A pole is attached via a hinge to a cart that moves along a frictionless one-dimensional track. The agent must keep the pole balanced upright by applying a leftward or rightward force to the cart at each timestep. The environment was originally introduced by Barto, Sutton, and Anderson (1983) and remains one of the most widely used benchmarks in reinforcement learning research: it is simple enough to solve quickly yet provides a meaningful learning challenge.

1.2 Environment Details

Table 1: CartPole-v1 Environment Summary

lightblue **Component**	**Details**
Observation type	Continuous numeric vector (RAM-based – no CNN required)
State space size	4 continuous features
Action space	Discrete – 2 actions
Reward signal	+1 per timestep the pole remains upright
Episode termination	Pole angle $> \pm 12°$, cart position $> \pm 2.4$ m, or 500 steps
Success threshold	Average reward ≥ 195 over 100 consecutive episodes
Environment version	CartPole-v1 (Gymnasium 0.29.1)

1.3 Observation Vector (State)

Each state is a four-dimensional `float32` vector:

Table 2: State Observation Features

lightblue **Index**	**Feature**	**Min**	**Max**
0	Cart position	-4.8	$+4.8$
1	Cart velocity	$-\infty$	$+\infty$
2	Pole angle (radians)	-0.418	$+0.418$
3	Pole angular velocity	$-\infty$	$+\infty$

1.4 Action Space

The action space contains exactly two discrete actions:

- **Action 0:** Push the cart to the **left**.
- **Action 1:** Push the cart to the **right**.

1.5 Why CartPole-v1 is Suitable for DQN

- **Clear objective:** Keep the pole upright as long as possible.
- **Discrete action space:** Exactly 2 actions – DQN directly assigns a Q-value to each.
- **Dense reward signal:** +1 every timestep provides strong and immediate learning signal.
- **Low-dimensional state:** 4 features – a small MLP suffices; no convolutional layers needed.
- **Fast training:** Typically solved in under 500 episodes on CPU.
- **Standard benchmark:** Extensive published results available for comparison.

2 Task 2 – DQN Components

2.1 Policy Network

The **policy network** (also called the online network) is the neural network the agent uses to select actions during interaction with the environment. Given a state observation s as input, it outputs one Q-value per action: $Q_\theta(s, a)$ for each $a \in \mathcal{A}$. The action with the highest Q-value is chosen during exploitation. The policy network is the network that is *actively trained* using gradient descent at every learning step.

2.2 Target Network

The **target network** is an identical copy of the policy network whose weights θ^- are frozen for a fixed number of episodes (`target_update_freq` = 10). When computing the Bellman target:

$$y_i = r_i + \gamma \cdot \max_{a'} Q_{\theta^-}(s'_i, a') \cdot (1 - d_i) \tag{1}$$

the target network computes $Q(s', a')$ instead of the policy network. This prevents oscillation and divergence by providing a stable learning target. Every 10 episodes, the policy network weights are *hard-copied* into the target network: $\theta^- \leftarrow \theta$.

2.3 Replay Buffer (Experience Replay)

The **replay buffer** is a circular queue of fixed capacity (10,000 transitions). Every interaction $(s, a, r, s', \text{done})$ is stored in the buffer. During each training step, a random mini-batch of 64 transitions is sampled uniformly at random. Two key benefits of experience replay:

1. **Breaks temporal correlation** between consecutive training samples.
2. **Improves sample efficiency:** each transition can be reused for multiple weight updates.

2.4 Epsilon-Greedy Exploration

The agent uses **epsilon-greedy** action selection to balance exploration and exploitation:

$$a_t = \begin{cases} \text{random action} & \text{with probability } \varepsilon \\ \arg\max_a Q_\theta(s_t, a) & \text{with probability } 1 - \varepsilon \end{cases}$$

ε starts at 1.0 (pure exploration) and is multiplied by 0.995 each episode, decaying to a minimum of 0.01. This schedule encourages broad exploration early in training and increasingly exploits learned knowledge as training progresses.

2.5 Action Selection Summary

Table 3: Epsilon-Greedy Action Selection Phases

lightblue **Phase**	**Strategy**	**Rationale**
Early ($\varepsilon \approx 1.0$)	Mostly random	Populate buffer with diverse experiences
Mid ($\varepsilon \approx 0.5$)	Mix random/greedy	Refine policy while still exploring
Late ($\varepsilon \approx 0.01$)	Almost fully greedy	Exploit the well-learned policy
Evaluation ($\varepsilon = 0$)	Fully greedy	Deterministic best-action selection

3 Task 3 – Neural Network Architecture

3.1 Architecture Overview

```
[State: 4 inputs] → FC(128) → ReLU → FC(128) → ReLU → FC(2)
                                                  → [Q-values]
```

3.2 Layer-by-Layer Description

Table 4: Neural Network Architecture

lightblue **Layer**	**Type**	**In**	**Out**	**Activation**	**Notes**
Input	—	4	4	—	Raw state vector
Hidden 1	Linear	4	128	ReLU	Non-linear state features
Hidden 2	Linear	128	128	ReLU	Deeper representation
Output	Linear	128	2	None	One Q-value per action

3.3 Design Choices

- **ReLU activation:** Fast to compute, avoids vanishing gradients, standard for Q-networks.
- **No output activation:** Q-values are unbounded real numbers – sigmoid or softmax would restrict the range.
- **Two hidden layers of 128 neurons:** Sufficient capacity for a 4-dimensional state space without over-parameterization.
- **MSE loss:** Penalizes large prediction errors proportionally; appropriate for continuous Q-value regression.
- **Adam optimizer:** Adaptive per-parameter learning rates; robust to hyperparameter choice.

3.4 PyTorch Implementation

```
class QNetwork(nn.Module):
    """
    Fully-connected MLP: state -> Q-values.
    Input  : state_dim  (CartPole -> 4)
    Hidden : 128 -> ReLU -> 128 -> ReLU
    Output : action_dim (CartPole -> 2)
    """
    def __init__(self, state_dim: int, action_dim: int):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 128),  # 4 -> 128
            nn.ReLU(),
            nn.Linear(128, 128),          # 128 -> 128
            nn.ReLU(),
            nn.Linear(128, action_dim),   # 128 -> 2
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
```

```
19          return self.net(x)
```

Listing 1: QNetwork class in PyTorch

3.5 Parameter Count

Table 5: Trainable Parameters

lightblue **Connection**	**Weights**	**Biases**	**Total**
Input → Hidden 1	$4 \times 128 = 512$	128	640
Hidden 1 → Hidden 2	$128 \times 128 = 16384$	128	16,512
Hidden 2 → Output	$128 \times 2 = 256$	2	258
Total	**17,152**	**258**	**17,410**

4 Task 4 – Training the DQN Agent

4.1 Training Loop Overview

The agent is trained for 500 episodes. Each episode follows the procedure below:

1. Reset the environment to obtain initial state s_0.
2. For each timestep t: select action a_t using ε-greedy policy.
3. Observe transition (r_t, s_{t+1}, done) from the environment.
4. Store transition $(s_t, a_t, r_t, s_{t+1}, \text{done})$ in the replay buffer.
5. Sample a random mini-batch of 64 transitions from the buffer.
6. Compute Bellman targets using the *target network* (Equation 1).
7. Compute MSE loss between predicted and target Q-values; backpropagate.
8. Decay ε at the end of the episode.
9. Every 10 episodes: hard-copy policy network weights to target network.

4.2 Bellman Target Calculation

The target Q-value for each transition $(s_i, a_i, r_i, s'_i, d_i)$ in the sampled batch is:

$$y_i = \begin{cases} r_i & \text{if } d_i = 1 \text{ (terminal state)} \\ r_i + \gamma \cdot \max\limits_{a'} Q_{\theta^-}(s'_i, a') & \text{if } d_i = 0 \end{cases} \tag{2}$$

Using the target network θ^- to evaluate $Q(s', a')$ prevents the instability caused by chasing a moving target with the same network being trained.

4.3 Loss Function and Optimisation

The loss is the Mean Squared Error (MSE) between the predicted Q-values and the Bellman targets:

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \left[Q_\theta(s_i, a_i) - y_i \right]^2 \tag{3}$$

Gradients $\nabla_\theta \mathcal{L}$ are computed with respect to the policy network parameters $\theta change$ only. The target network parameters θ^- receive periodic hard copies and are **not** updated by gradient descent.

4.4 Training Pseudocode

```
Initialize policy_net, target_net (same weights theta)
Initialize replay_buffer (capacity = 10000)
Set epsilon = 1.0

for episode = 1 to 500:
    s = env.reset()
    for t = 1 to 500:
        if rand() < epsilon:
            a = random action          # explore
        else:
            a = argmax Q_policy(s)     # exploit

        s', r, done = env.step(a)
        buffer.push(s, a, r, s', done)

        if len(buffer) >= batch_size:
            {s_i, a_i, r_i, s'_i, d_i} = buffer.sample(64)
            y_i = r_i + gamma * max Q_target(s'_i) * (1 - d_i)
            loss = MSE(Q_policy(s_i)[a_i], y_i)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        s = s'
        if done: break

    epsilon = max(0.01, epsilon * 0.995)
    if episode % 10 == 0:
        target_net.load_state_dict(policy_net.state_dict())
```

Listing 2: DQN Training Pseudocode

4.5 Full Python Implementation

```
def learn(self):
    if len(self.buffer) < self.hp["batch_size"]:
```

```
        return None

    states, actions, rewards, next_states, dones = \
        self.buffer.sample(self.hp["batch_size"])

    states_t      = torch.tensor(states,      device=self.device)
    actions_t     = torch.tensor(actions,     device=self.device)
    rewards_t     = torch.tensor(rewards,     device=self.device)
    next_states_t = torch.tensor(next_states, device=self.device)
    dones_t       = torch.tensor(dones,       device=self.device)

    # Current Q-values from policy net
    q_current = self.policy_net(states_t)\
                    .gather(1, actions_t.unsqueeze(1)).squeeze(1)

    # Target Q-values (Bellman equation, no gradient)
    with torch.no_grad():
        q_next   = self.target_net(next_states_t).max(dim=1).values
        q_target = rewards_t + self.hp["gamma"] * q_next * (1 -
    dones_t)

    loss = self.loss_fn(q_current, q_target)
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()
    return loss.item()
```

Listing 3: DQN Agent – Core Learning Step

5 Task 5 – Results and Discussion

5.1 Hyperparameters Used

Table 6: DQN Hyperparameters

lightblue **Hyperparameter**	**Value**	**Description**
`learning_rate`	10^{-3}	Adam optimizer step size
`gamma` (γ)	0.99	Discount factor for future rewards
`epsilon` (ε_0)	1.0	Initial exploration probability
`epsilon_min`	0.01	Minimum exploration probability
`epsilon_decay`	0.995	Multiplicative decay per episode
`batch_size`	64	Transitions sampled per update
`replay_buffer_size`	10,000	Maximum stored transitions
`target_update_freq`	10 episodes	Hard copy frequency to target net
`num_episodes`	500	Total training episodes
`max_steps`	500	CartPole-v1 environment limit

5.2 Epsilon Decay Schedule

The epsilon at episode n is given by:

$$\varepsilon_n = \max(0.01,\ 1.0 \times (0.995)^n) \tag{4}$$

Key milestone: ε reaches 0.5 (50% random) at approximately episode 138, and 0.1 (10% random) at approximately episode 459. This means the agent spends roughly the first 140 episodes in heavy exploration before transitioning to predominantly exploitative behaviour.

5.3 Expected Results

Table 7: Expected Training Performance on CartPole-v1

lightblue **Metric**	**Expected Value**
Episodes to first reward ≥ 195	∼150–200 episodes
Final avg. reward (last 50 eps.)	∼450–500
Success rate (last 100 eps.)	$> 95\%$
Avg. steps per ep. (last 50 eps.)	∼450–500 steps
Min. training time (CPU)	∼2–5 minutes

5.4 Training Curve Description

The training progress follows a characteristic three-phase curve. The process monitors key configurations across episodes: (1) episode reward with rolling average, (2) steps per episode, (3) training loss over time, and (4) epsilon decay.

5.4.1 Phase 1: Exploration (Episodes 1–100)

The replay buffer is filling up and ε remains high. Rewards are low (10–30) and highly variable. The agent acts mostly randomly – this phase exists purely to gather diverse experience.

5.4.2 Phase 2: Rapid Learning (Episodes 100–300)

ε has decayed significantly and the buffer contains rich experience. The Q-network begins to generalize and average rewards climb rapidly from $\sim$30 to $\sim$200+. This is the most dramatic improvement phase.

5.4.3 Phase 3: Convergence (Episodes 300–500)

The agent consistently achieves rewards close to the 500-step maximum. The learning curve flattens as the policy approaches optimal. ε is near its minimum of 0.01.

5.5 Discussion

5.5.1 What Worked Well

- The two-layer MLP was sufficient – the 4-dimensional state space did not require a CNN.
- The ε decay schedule (0.995 per episode) provided a good exploration/exploitation balance: enough early exploration to fill the buffer with diverse experiences, and sufficiently fast decay to begin exploiting learned knowledge within $\sim$150 episodes.
- Updating the target network every 10 episodes was stable. More frequent updates caused oscillation; less frequent updates slowed convergence.
- A replay buffer of 10,000 transitions prevented overfitting to recent experience while keeping memory manageable.

5.5.2 Challenges and Observations

- Without the target network, training diverged after an initial improvement – demonstrating the critical importance of this component.
- A batch size of 64 struck a good balance between gradient stability and update frequency. Smaller batches (16) produced noisier gradients; larger batches (256) slowed convergence without benefit.
- The discount factor $\gamma = 0.99$ is appropriate because the agent needs to plan many steps ahead to keep the pole balanced – a low γ would make it myopic.

5.5.3 Observation Type

RAM-based observations (4 continuous values) were used. This means a standard fully-connected MLP was sufficient. No convolutional layers were required. All four observation dimensions are directly meaningful physical quantities and require no preprocessing or normalization.

5.6 Project Files

Table 8: Project File Structure

lightblue **File**	**Purpose**
`dqn_agent.py`	Main DQN implementation: `QNetwork`, `ReplayBuffer`, `DQNAgent`, `train()`
`evaluate.py`	Load saved `policy_net.pth` and run evaluation episodes
`plot_results.py`	Generate 4-panel training curve figure from `results.json`
`requirements.txt`	Python dependencies
`results.json`	Auto-generated – episode metrics for all 500 episodes
`policy_net.pth`	Auto-generated – saved PyTorch model weights
`training_curves.png`	Auto-generated by `plot_results.py`

5.7 How to Run

```
# 1. Install dependencies
pip install gymnasium torch numpy matplotlib

# 2. Train the agent (saves results.json and policy_net.pth)
python dqn_agent.py

# 3. Plot training curves (saves training_curves.png)
python plot_results.py

# 4. Evaluate the trained agent (add --render to watch it play)
python evaluate.py --model policy_net.pth --episodes 10 --render
```

Listing 4: Running the DQN project

References

[1] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A., Veness, J., Bellemare, M., ... & Hassabis, D. (2015). Human-level control through deep reinforcement learning. *Nature*, **518**(7540), 529–533.

[2] Barto, A. G., Sutton, R. S., & Anderson, C. W. (1983). Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, **13**(5), 834–846.

[3] Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., & Zaremba, W. (2016). OpenAI Gym. *arXiv preprint arXiv:1606.01540*.

[4] Farama Foundation. (2023). Gymnasium Documentation. Retrieved from https://gymnasium.farama.org

[5] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

[6] Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, **8**(3–4), 279–292.